

# 視訊處理作業報告

HW4

Mixed resolution video  
coding

授課教授：柳金章

學 生：楊憲閔

學 號：613410047

Due date：2025/06/09

Date hand in：2025/06/14

# 目錄

|                               |    |
|-------------------------------|----|
| Technical description.....    | 3  |
| Experimental results.....     | 7  |
| Discussions .....             | 8  |
| References and Appendix ..... | 14 |

# Technical description

## 1. Mixed Resolution Video Coding

傳統的視訊編碼(如 H.264)通常採用固定大小(例如 16x16)的 macroblock 進行視訊壓縮、運動估計與補償。而在高效率視訊編碼(HEVC)中,則引入了可變大小的編碼單元(CU, Coding Unit),最大尺寸為 64x64,可根據畫面內容進一步細分為 Prediction Unit (PU) 與 Transform Unit (TU)。這種可調區塊大小的設計,使得編碼器能針對畫面中的重要區域使用高解析區塊保留細節,而對背景等次要區域則使用低解析度區塊以節省位元率,進而達到壓縮與品質間的平衡。

Mixed Resolution Video Coding 即是建立在這樣的架構基礎上,透過將部分 frame 或區塊以較低解析度編碼,產生混合解析度的視訊序列(Mixed Resolution Sequence),再透過重建方法將其還原為一致解析度的高畫質影像。

## 2. Image Interpolation

影像插值(Image Interpolation)是一種常見的影像處理方法,用以估計未知像素的強度值。方法是利用該像素鄰近區域已知像素

值來推算，確保圖像放大過程中畫面連續且自然。

在本次作業中，首先針對解析度較低的幀進行雙線性插值 (Bilinear Interpolation)，使其還原至與 HR frame 相同解析度，作為第一步的 baseline 重建方法。

### 3. Motion Compensation

為了進一步提升重建品質，本作業採用 Motion Compensation 技術，透過運動估計 (Motion Estimation) 尋找幀與幀之間的對應區塊資訊，用以修復 interpolation 無法精確還原的細節。

本次實作了 Full Search：暴力搜尋參考幀中的所有可能區塊，選擇差異最小 (MAD) 的作為最佳匹配區塊，重建目標區域。透過與 original 相同解析度的作為高解析關鍵幀，進行插值與運動補償還原。

# Experimental results

## 1. 程式執行流程：

- (1) 確保已安裝相關 module，本次作業使用 module 如下所示：

```
import cv2
import numpy as np
import os, glob
from skimage.metrics import peak_signal_noise_ratio as psnr
```

圖(1) 會使用到的 module

- (2) 進到作業的目錄底下，會看到 Hw4\_test sequences 資料夾、HW4.py 檔及此 pdf 檔。點右鍵按在終端中開啟，輸入 python main.py，程式即開始執行。
- (3) 程式會讀取 sequence 的資料並進行兩種方法還原 resolution，第一種為單純 Interpolation，為使用雙線性插值進行上採樣；第二種為第一種方法加入 Full Search 進行 Motion Compensation。並印出每個 frame 在不同方法中的 psnr 值，且輸出於各自的輸出資料夾中。

## 2. 程式執行結果：

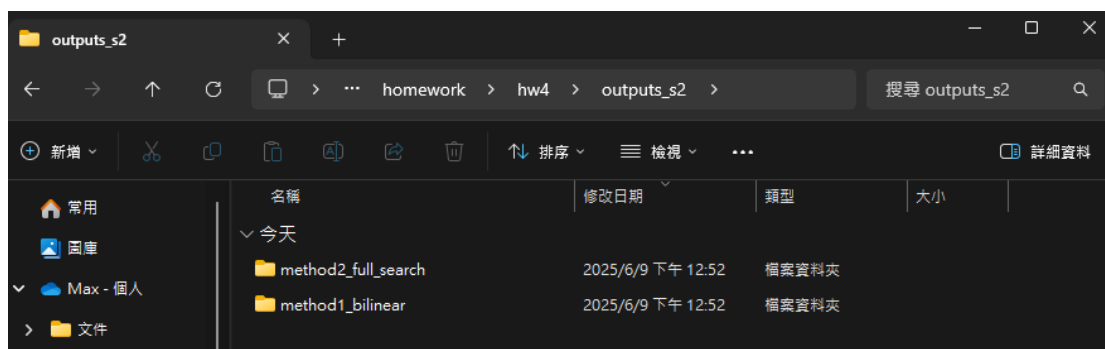
會印出各種方法 reconstruct 後與 original image 之間的 PSNR 值，並且於固定的位置儲存處理過後的 frame，例如說 s1 sequence 處理完就會放置於 output\_s1 中(又因有兩種方法，因

此底下會有兩種 method 的資料夾)。

因印出內容過多，將由例圖代為說明。

```
=== sequence: s1 ===
PSNR values for each method:
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\skimage\metrics\simple_metrics.py:168: RuntimeWarning: divide by zero encountered in scalar divide
  return 10 * np.log10((data_range**2) / err)
frame 0: Only Bilinear interpolation: inf, Full Search: inf
frame 1: Only Bilinear interpolation: 27.603857177188416, Full Search: 28.224378217007782
frame 2: Only Bilinear interpolation: 27.451504939461966, Full Search: 28.232579795082245
frame 3: Only Bilinear interpolation: 27.47348336421355, Full Search: 28.190444388182733
frame 4: Only Bilinear interpolation: inf, Full Search: inf
frame 5: Only Bilinear interpolation: 27.5788908868916, Full Search: 28.70740186666611
frame 6: Only Bilinear interpolation: 27.62995231415012, Full Search: 28.814070756172843
frame 7: Only Bilinear interpolation: 27.687445403184174, Full Search: 28.957053364965795
frame 8: Only Bilinear interpolation: inf, Full Search: inf
frame 9: Only Bilinear interpolation: 27.523107341079417, Full Search: 28.468930247119687
frame 10: Only Bilinear interpolation: 27.536736019180637, Full Search: 28.557486103961747
frame 11: Only Bilinear interpolation: 27.521777639308468, Full Search: 28.173758852848874
frame 12: Only Bilinear interpolation: inf, Full Search: inf
frame 13: Only Bilinear interpolation: 27.501520740267722, Full Search: 27.854518157065055
frame 14: Only Bilinear interpolation: 27.530010140818316, Full Search: 27.90170491330075
frame 15: Only Bilinear interpolation: 27.530605528457976, Full Search: 27.922125931764473
frame 16: Only Bilinear interpolation: inf, Full Search: inf
frame 17: Only Bilinear interpolation: 27.48346881326342, Full Search: 28.514011901698085
frame 18: Only Bilinear interpolation: 27.320455903638035, Full Search: 28.232178638533284
frame 19: Only Bilinear interpolation: 27.267111673025695, Full Search: 28.211428731120446
frame 20: Only Bilinear interpolation: inf, Full Search: inf
frame 21: Only Bilinear interpolation: 27.392969246737437, Full Search: 27.46158097935194
frame 22: Only Bilinear interpolation: 27.500880412354274, Full Search: 27.861896510372006
frame 23: Only Bilinear interpolation: 27.637002451734972, Full Search: 28.168787344517362
frame 24: Only Bilinear interpolation: inf, Full Search: inf
```

圖(2) 各種方法與原圖比較之 PSNR (以 s1 為例)，inf 表示本來就是高解析度，因此會與 original 一模一樣



圖(3) 各種方法會放置於對應的資料夾底下  
(裡面有重建後的 frame)

表(1) s1 在不同方法中的 PSNR (db)，取到小數點後第 2 位

|      | 1. bmp | 25. bmp | 50. bmp | 75. bmp | 87. bmp |
|------|--------|---------|---------|---------|---------|
| 方法 1 | 27.60  | 27.61   | 28.04   | 28.09   | 28.40   |

|      |       |       |       |       |       |
|------|-------|-------|-------|-------|-------|
| 方法 2 | 28.22 | 28.21 | 28.49 | 28.60 | 28.95 |
|------|-------|-------|-------|-------|-------|

表(2) s2 在不同方法中的 PSNR (db)，取到小數點後第 2 位

|      | 1. bmp | 15. bmp | 27. bmp |
|------|--------|---------|---------|
| 方法 1 | 26.24  | 27.08   | 26.59   |
| 方法 2 | 26.74  | 26.69   | 26.22   |

# Discussions

本次作業主要比較了兩種影像重建方式：一為單純使用雙線性插值 (Interpolation)，另一為結合 Full Search 的 Motion Compensation 技術來進行更精確的重建。

從實驗結果來看，對於 s1 這組影像序列，加入 Motion Compensation 的方法在每個測試影格的 PSNR 值皆高於單純插值，顯示出透過運動補償可以有效地修復插值無法還原的細節，提升重建影像的品質。這符合預期，因為 s1 可能包含較明顯的畫面運動，透過 Motion Estimation 可以更準確地預測區塊變化，降低失真。

然而，在 s2 序列中，部分影格的結果卻顯示 Motion Compensation 不一定優於單純插值，甚至在某些影格的 PSNR 表現較差。這可能與該序列的特性有關，例如影像之間的變化較小，或幀與幀之間的運動資訊不明顯，導致運動估計無法提供準確的對應區塊，反而引入預測錯誤，進一步影響重建品質。

綜合來說，Motion Compensation 的效能與影像本身的特性有密切關聯。在高動態影像中，其效果顯著優於單純插值；而在低動態或靜態影像中，則需謹慎使用，以避免過度預測造成的失真。未



來若能搭配運動強度分析來動態決定是否使用 Motion Compensation，有機會進一步提升重建效能與品質。

# References and Appendix

<https://zh.wikipedia.org/zh->

[tw/%E9%AB%98%E6%95%88%E7%8E%87%E8%A7%86%E9%A2%91%E7%BC%96%E7%A0%81](https://zh.wikipedia.org/zh-tw/%E9%AB%98%E6%95%88%E7%8E%87%E8%A7%86%E9%A2%91%E7%BC%96%E7%A0%81)

<https://chatgpt.com>